

Reviewer Pack — Submodular Coverage Deficit in Monotone DNF for k-CLIQUE

Entry d4c1f94494d9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 19:10:11 UTC

Submodular Coverage Deficit in Monotone DNF for k-CLIQUE

Entry ID: d4c1f94494d9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 19:10:11 UTC

1. Conjecture

Field A (mathematical branch): Combinatorics (Submodular Functions) **Field B** (complexity object): Monotone Circuit Complexity

Statement:

Let $\mu(D) = \sum |T_i| - \sum |T_i \cap T_j|$ for a DNF D with terms T_i . For any DNF of size $\text{poly}(n)$, $\mu(D) \leq O(\log n)$. For the k-CLIQUE indicator function, $\mu(D) \geq \Omega(n^{\{1/2\}})$ when $k \geq 3$.

Rationale (proposer’s reasoning):

Submodular coverage deficit captures term overlaps in DNFs. For k-CLIQUE, the exponential number of terms creates large overlaps, making $\mu(D)$ scale with $n^{\{1/2\}}$. This links combinatorial submodularity to monotone circuit lower bounds.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b75c75811625cebe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_dnf(n, k):
        terms = []
        for _ in range(2**n):
            term = [random.randint(0, 1) for _ in range(n)]
            if sum(term) >= k:
                terms.append(term)
        return terms

    def set_intersection_size(A, B):
        return len([i for i in range(len(A)) if A[i] == 1 and B[i] == 1])

    n = random.randint(5, 40)
    D = generate_dnf(n, 3) # k-CLIQUE with k=3
    instances_tested = len(D)

    mu_D = sum(len(term) for term in D)
    for i in range(instances_tested):
        for j in range(i + 1, instances_tested):
            mu_D -= set_intersection_size(D[i], D[j])

    conjecture_holds = False
    counterexample = ""

    if n <= 40:
        if mu_D >= math.sqrt(n):
            conjecture_holds = True
        else:
            counterexample = "mu(D) < sqrt(n)"
    else:
        counterexample = "mapping_undefined"

    return {
        "metric_name": "mu(D)",
        "metric_value": mu_D,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_mu_D = sum(r["metric_value"] for r in results) / len(results)
    std_mu_D = math.sqrt(sum((r["metric_value"] - mean_mu_D)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_mu_D} std={std_mu_D} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mu(D) < sqrt(n)\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing results; insufficient data to evaluate conjecture. | next:
Run test with extended timeout and debug crash logs

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111266
2	propose	ollama_remote	qwen3:8b	0	0	119793
3	preregistration	ollama_remote	qwen3:8b	0	0	24070
4	novelty	ollama_remote	qwen3:8b	0	0	20728
5	novelty	ollama_remote	qwen3:8b	0	0	16727

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11017
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7160
8	judge	ollama_remote	qwen3:8b	0	0	73674

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 384436 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/d4c1f94494d9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d4c1f94494d9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d4c1f94494d9.tar.gz (if generated)